

FLIGHT BOOKS AUTO-UPDATE PIPELINE — PROJECT SCHEMA

RSS → Firecrawl → OpenAI → Supabase + MCP Chat Interface

Stack: Next.js 14 · Supabase · Firecrawl · OpenAI GPT-4o · MCP (FastMCP) · Vercel Cron

PROJECT STRUCTURE

```
flight-books-pipeline/
├── app/
│   ├── api/
│   │   └── cron/
│   │       └── route.ts                # Vercel Cron endpoint (POST)
├── lib/
│   ├── rss/
│   │   ├── fetchFeeds.ts              # Fetch + parse RSS XML
│   │   └── diffFeeds.ts               # Compare against last-seen GUIDs
│   ├── scraper/
│   │   └── firecrawl.ts               # Firecrawl API wrapper
│   ├── ai/
│   │   └── extractUpdates.ts          # OpenAI GPT-4o structured extraction
│   ├── db/
│   │   ├── flightBooks.ts             # Read/write flight book records
│   │   └── feedState.ts               # Read/write last-seen feed state
│   └── mcp/
│       └── server.ts                  # MCP server exposing DB as tools
├── mcp-server/
│   ├── index.ts                       # Standalone MCP server entry point
│   └── tools/
│       ├── searchFlightBooks.ts       # MCP tool: search flight books
│       ├── getRecentUpdates.ts        # MCP tool: get recent updates
│       └── getUpdateHistory.ts        # MCP tool: get update history log
├── supabase/
│   └── migrations/
│       ├── 001_flight_books.sql       # Flight books table
│       ├── 002_feed_state.sql         # RSS feed state tracker
│       └── 003_update_log.sql         # Audit log of all updates
└── .env.local
```

```
└─ vercel.json                                # Cron schedule config
└─ package.json
```

DATABASE SCHEMA (Supabase)

Table: `flight_books`

Stores the live flight book records that get updated by the pipeline.

```
-- 001_flight_books.sql
CREATE TABLE flight_books (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       TEXT NOT NULL,
  category    TEXT,                                -- e.g. 'procedures', 'weather', 'ai
  content     TEXT NOT NULL,                        -- Full structured content (markdown
  source_url  TEXT,                                -- Originating source URL
  version     INTEGER DEFAULT 1,
  last_updated TIMESTAMPTZ DEFAULT NOW(),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_flight_books_category ON flight_books(category);
CREATE INDEX idx_flight_books_title ON flight_books USING gin(to_tsvector('englis
CREATE INDEX idx_flight_books_content ON flight_books USING gin(to_tsvector('engl
```

Table: `rss_feed_state`

Tracks the last-seen state of each RSS feed to enable diffing.

```
-- 002_feed_state.sql
CREATE TABLE rss_feed_state (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  feed_url    TEXT UNIQUE NOT NULL,
  feed_name   TEXT,
  last_seen_guids TEXT[] DEFAULT '{}',            -- Array of GUIDs seen in last run
  last_fetched TIMESTAMPTZ,
  last_item_pub TIMESTAMPTZ,                      -- pubDate of most recent item seen
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Table: `update_log`

Full audit trail of every update the pipeline has written.

```
-- 003_update_log.sql
CREATE TABLE update_log (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  flight_book_id    UUID REFERENCES flight_books(id),
  source_feed_url   TEXT NOT NULL,
  source_item_url   TEXT NOT NULL,
  source_item_guid  TEXT NOT NULL,
  previous_content  TEXT,                -- Snapshot before update
  new_content       TEXT,                -- Content after update
  openai_response   JSONB,              -- Raw OpenAI structured output
  status            TEXT DEFAULT 'applied', -- 'applied' | 'skipped' | 'failed'
  applied_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_update_log_book ON update_log(flight_book_id);
CREATE INDEX idx_update_log_status ON update_log(status);
CREATE INDEX idx_update_log_date ON update_log(applied_at DESC);
```

ENVIRONMENT VARIABLES

```
# .env.local

# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key

# Firecrawl
FIRECRAWL_API_KEY=your-firecrawl-key

# OpenAI
OPENAI_API_KEY=your-openai-key

# Cron Security (generate a random secret)
CRON_SECRET=your-random-secret-string

# RSS Feeds (comma-separated)
RSS_FEED_URLS=https://feed1.example.com/rss,https://feed2.example.com/rss
```

VERCEL CRON CONFIG

```
// vercel.json
{
  "crons": [
    {
      "path": "/api/cron",
      "schedule": "0 */6 * * *"
    }
  ]
}
```

Runs every 6 hours. Adjust `schedule` to your preferred frequency. The route is protected by `CRON_SECRET` to prevent unauthorised triggers.

PIPELINE MODULES

1. RSS Fetcher — `lib/rss/fetchFeeds.ts`

```
import Parser from 'rss-parser'

const parser = new Parser()

export interface FeedItem {
  guid: string
  title: string
  link: string
  pubDate: string
  contentSnippet?: string
}

export async function fetchFeed(feedUrl: string): Promise<FeedItem[]> {
  const feed = await parser.parseURL(feedUrl)
  return feed.items.map(item => ({
    guid: item.guid || item.link || '',
    title: item.title || '',
    link: item.link || '',
    pubDate: item.pubDate || '',
    contentSnippet: item.contentSnippet || ''
  }))
}
```

Install: `npm install rss-parser`

2. Feed Differ — `lib/rss/diffFeeds.ts`

```
import { supabase } from '@lib/db/client'
import { FeedItem } from './fetchFeeds'

export async function getNewItems(
  feedUrl: string,
  currentItems: FeedItem[]
): Promise<FeedItem[]> {
  // Load last-seen GUIDs from DB
  const { data } = await supabase
    .from('rss_feed_state')
    .select('last_seen_guids')
    .eq('feed_url', feedUrl)
    .single()

  const lastSeenGuids: string[] = data?.last_seen_guids || []
  const currentGuids = currentItems.map(i => i.guid)

  // Return only items with unseen GUIDs
  const newItems = currentItems.filter(
    item => !lastSeenGuids.includes(item.guid)
  )

  // Save updated state back to DB
  await supabase
    .from('rss_feed_state')
    .upsert({
      feed_url: feedUrl,
      last_seen_guids: currentGuids,
      last_fetched: new Date().toISOString(),
      last_item_pub: currentItems[0]?.pubDate
    }, { onConflict: 'feed_url' })

  return newItems
}
```

3. Firecrawl Scraper — `lib/scraper/firecrawl.ts`

```
import FirecrawlApp from '@mendable/firecrawl-js'
```

```

const firecrawl = new FirecrawlApp({ apiKey: process.env.FIRECRAWL_API_KEY! })

export async function scrapeUrl(url: string): Promise<string> {
  const result = await firecrawl.scrapeUrl(url, {
    formats: ['markdown'],
    onlyMainContent: true
  })

  if (!result.success) {
    throw new Error(`Firecrawl failed for ${url}: ${result.error}`)
  }

  return result.markdown || ''
}

```

Install: `npm install @mendable/firecrawl-js`

4. OpenAI Extractor — `lib/ai/extractUpdates.ts`

This is the intelligence layer. It receives both the scraped content AND the current flight book record, then returns a structured update decision.

```

import OpenAI from 'openai'
import { zodResponseFormat } from 'openai/helpers/zod'
import { z } from 'zod'

const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY! })

// Define the structured output schema
const UpdateDecision = z.object({
  should_update: z.boolean(),
  reason: z.string(),
  updated_content: z.string().optional(), // Only present if should_update = true
  affected_sections: z.array(z.string()), // Which parts of the book changed
  confidence: z.enum(['high', 'medium', 'low'])
})

export type UpdateDecision = z.infer<typeof UpdateDecision>

export async function extractUpdate(params: {
  scrapedContent: string
  currentFlightBook: { id: string; title: string; content: string }
  sourceUrl: string
}): Promise<UpdateDecision> {

```

```

const response = await client.beta.chat.completions.parse({
  model: 'gpt-4o',
  response_format: zodResponseFormat(UpdateDecision, 'update_decision'),
  messages: [
    {
      role: 'system',
      content: `You are an aviation data analyst. Your job is to compare
newly scraped content from an aviation RSS feed against an existing flight
book record, and determine if the flight book needs updating.

Rules:
- Only recommend an update if the new content contains materially different
  or new information relevant to the flight book topic.
- If updating, rewrite the full content field incorporating the new information.
- Preserve all existing accurate information. Do not remove valid content.
- Be conservative: if unsure, set should_update = false and confidence = low.`
    },
    {
      role: 'user',
      content: `FLIGHT BOOK RECORD:
Title: ${params.currentFlightBook.title}
Current Content:
${params.currentFlightBook.content}

---

NEW SCRAPED CONTENT FROM: ${params.sourceUrl}
${params.scrapedContent}

---

Should this flight book record be updated based on the new content?`
    }
  ]
})

return response.choices[0].message.parsed!
}

```

Install: npm install openai zod

5. Database Helpers — lib/db/flightBooks.ts

```

import { supabase } from './client'
import { UpdateDecision } from '@lib/ai/extractUpdates'

export async function getAllFlightBooks() {
  const { data, error } = await supabase
    .from('flight_books')
    .select('*')
    .order('category')
  if (error) throw error
  return data
}

export async function applyUpdate(params: {
  flightBookId: string
  newContent: string
  previousContent: string
  sourceItemUrl: string
  sourceFeedUrl: string
  sourceItemGuid: string
  openaiResponse: UpdateDecision
}) {
  // Update the flight book
  const { error: updateError } = await supabase
    .from('flight_books')
    .update({
      content: params.newContent,
      last_updated: new Date().toISOString(),
    })
    .eq('id', params.flightBookId)

  if (updateError) throw updateError

  // Write to update_log (audit trail)
  const { error: logError } = await supabase
    .from('update_log')
    .insert({
      flight_book_id: params.flightBookId,
      source_feed_url: params.sourceFeedUrl,
      source_item_url: params.sourceItemUrl,
      source_item_guid: params.sourceItemGuid,
      previous_content: params.previousContent,
      new_content: params.newContent,
      openai_response: params.openaiResponse,
      status: 'applied'
    })
}

```

```
    if (logError) throw logError
  }
```

6. Cron Route — `app/api/cron/route.ts`

This is the main orchestrator. Vercel calls this on schedule.

```
import { NextRequest, NextResponse } from 'next/server'
import { fetchFeed } from '@lib/rss/fetchFeeds'
import { getNewItem } from '@lib/rss/diffFeeds'
import { scrapeUrl } from '@lib/scrapper/firecrawl'
import { extractUpdate } from '@lib/ai/extractUpdates'
import { getAllFlightBooks, applyUpdate } from '@lib/db/flightBooks'

export async function POST(req: NextRequest) {
  // Auth check
  const authHeader = req.headers.get('authorization')
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }

  const feedUrls = process.env.RSS_FEED_URLS!.split(',')
  const flightBooks = await getAllFlightBooks()
  const results = []

  for (const feedUrl of feedUrls) {
    try {
      // 1. Fetch feed
      const items = await fetchFeed(feedUrl)

      // 2. Diff - only new items
      const newItems = await getNewItem(feedUrl, items)
      console.log(`[${feedUrl}] ${newItems.length} new items`)

      for (const item of newItems) {
        try {
          // 3. Scrape the linked article
          const scrapedContent = await scrapeUrl(item.link)

          // 4. Compare against each flight book
          for (const book of flightBooks) {
            const decision = await extractUpdate({
              scrapedContent,
              currentFlightBook: book,
              sourceUrl: item.link
            })
          }
        } catch (error) {
          console.error(`Error scraping ${item.link}:`, error)
        }
      }
    } catch (error) {
      console.error(`Error fetching feed ${feedUrl}:`, error)
    }
  }

  return NextResponse.json({ results })
}
```

```

    })

    if (decision.should_update && decision.updated_content) {
      // 5. Write update + log it
      await applyUpdate({
        flightBookId: book.id,
        newContent: decision.updated_content,
        previousContent: book.content,
        sourceItemUrl: item.link,
        sourceFeedUrl: feedUrl,
        sourceItemGuid: item.guid,
        openaiResponse: decision
      })

      results.push({
        bookId: book.id,
        bookTitle: book.title,
        sourceUrl: item.link,
        status: 'updated',
        confidence: decision.confidence
      })
    }
  }
} catch (itemError) {
  console.error(`Error processing item ${item.link}:`, itemError)
  results.push({ sourceUrl: item.link, status: 'failed' })
}

} catch (feedError) {
  console.error(`Error processing feed ${feedUrl}:`, feedError)
}

}

return NextResponse.json({ processed: results.length, results })
}

```

MCP SERVER

The MCP server wraps your Supabase flight books DB so Claude can query it in a chat interface. Run it as a standalone Node process alongside your app.

mcp-server/index.ts — Entry Point

```
import { FastMCP } from 'fastmcp'
import { searchFlightBooksTool } from '../tools/searchFlightBooks'
import { getRecentUpdatesTool } from '../tools/getRecentUpdates'
import { getUpdateHistoryTool } from '../tools/getUpdateHistory'

const mcp = new FastMCP('Flight Books MCP Server')

mcp.addTool(searchFlightBooksTool)
mcp.addTool(getRecentUpdatesTool)
mcp.addTool(getUpdateHistoryTool)

mcp.start({ transportType: 'stdio' })
```

Install: `npm install fastmcp @supabase/supabase-js`

Tool 1 — `mcp-server/tools/searchFlightBooks.ts`

```
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  process.env.SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)

export const searchFlightBooksTool = {
  name: 'search_flight_books',
  description: 'Search flight book records by keyword or category. Use this to find flight information.',
  parameters: {
    type: 'object',
    properties: {
      query: {
        type: 'string',
        description: 'Search term or question (e.g. "landing minimums", "Class C", "VFR", "instrument", "weather", "procedures")',
      },
      category: {
        type: 'string',
        description: 'Optional: filter by category (e.g. "procedures", "weather", "instrument", "VFR", "Class C", "landing minimums")',
      },
    },
    required: ['query']
  },
  execute: async ({ query, category }: { query: string; category?: string }) => {
    let dbQuery = supabase
      .from('flight_books')
      .select('*')
      .text(searchTerm)`
```

```

        .select('id, title, category, content, last_updated')
        .textSearch('content', query, { type: 'websearch' })

    if (category) {
        dbQuery = dbQuery.eq('category', category)
    }

    const { data, error } = await dbQuery.limit(5)
    if (error) throw error

    return JSON.stringify(data, null, 2)
}
}

```

Tool 2 — `mcp-server/tools/getRecentUpdates.ts`

```

export const getRecentUpdatesTool = {
  name: 'get_recent_updates',
  description: 'Get a list of recent updates made to flight books by the pipeline',
  parameters: {
    type: 'object',
    properties: {
      days: {
        type: 'number',
        description: 'Number of days to look back (default: 7)'
      },
      limit: {
        type: 'number',
        description: 'Max number of results to return (default: 10)'
      }
    },
    required: []
  },
  execute: async ({ days = 7, limit = 10 }: { days?: number; limit?: number }) => {
    const since = new Date()
    since.setDate(since.getDate() - days)

    const { data, error } = await supabase
      .from('update_log')
      .select(`
        id,
        applied_at,
        source_item_url,
        status,

```

```

        openai_response,
        flight_books(title, category)
    `)
    .eq('status', 'applied')
    .gte('applied_at', since.toISOString())
    .order('applied_at', { ascending: false })
    .limit(limit)

    if (error) throw error

    return JSON.stringify(data, null, 2)
}
}

```

Tool 3 — mcp-server/tools/getUpdateHistory.ts

```

export const getUpdateHistoryTool = {
  name: 'get_update_history',
  description: 'Get the full update history for a specific flight book, including parameters: {
    type: 'object',
    properties: {
      flight_book_id: {
        type: 'string',
        description: 'UUID of the flight book'
      },
      limit: {
        type: 'number',
        description: 'Max number of history entries (default: 20)'
      }
    },
    required: ['flight_book_id']
  },
  execute: async ({ flight_book_id, limit = 20 }: { flight_book_id: string; limit
    const { data, error } = await supabase
      .from('update_log')
      .select('id, applied_at, source_item_url, previous_content, new_content, op
      .eq('flight_book_id', flight_book_id)
      .order('applied_at', { ascending: false })
      .limit(limit)

    if (error) throw error

    return JSON.stringify(data, null, 2)
  }
}

```

```
}  
}
```

CLAUDE DESKTOP MCP CONFIG

To connect the MCP server to Claude Desktop for the chat interface, add this to your `claude_desktop_config.json`:

```
{  
  "mcpServers": {  
    "flight-books": {  
      "command": "node",  
      "args": ["/absolute/path/to/flight-books-pipeline/mcp-server/index.js"],  
      "env": {  
        "SUPABASE_URL": "https://your-project.supabase.co",  
        "SUPABASE_SERVICE_ROLE_KEY": "your-service-role-key"  
      }  
    }  
  }  
}
```

Claude Desktop config location:

- **macOS:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

BUILD ORDER FOR CURSOR

Implement in this exact sequence to avoid dependency errors:

Phase 1 — Database

- [] Run migrations 001, 002, 003 in Supabase SQL editor
- [] Create `lib/db/client.ts` (Supabase client singleton)

Phase 2 — Pipeline Core

- [] `lib/rss/fetchFeeds.ts`
- [] `lib/rss/diffFeeds.ts`
- [] `lib/scrapper/firecrawl.ts`
- [] `lib/ai/extractUpdates.ts`
- [] `lib/db/flightBooks.ts`

```
[ ] lib/db/feedState.ts
```

Phase 3 – Cron Orchestrator

```
[ ] app/api/cron/route.ts
[ ] vercel.json
[ ] Test manually with POST + Bearer token
```

Phase 4 – MCP Server

```
[ ] mcp-server/index.ts
[ ] mcp-server/tools/searchFlightBooks.ts
[ ] mcp-server/tools/getRecentUpdates.ts
[ ] mcp-server/tools/getUpdateHistory.ts
[ ] Connect to Claude Desktop via config
```

Phase 5 – Test & Validate

```
[ ] Trigger cron manually, verify update_log entries
[ ] Ask Claude Desktop: "What was updated in the last 7 days?"
[ ] Ask Claude Desktop: "Search flight books for landing procedures"
```

COST ESTIMATES

Operation	Cost indicator
Firecrawl scrape	~\$0.002/page (only new RSS items)
GPT-4o per book comparison	~\$0.01–0.03 per call
Supabase	Free tier sufficient to start
Vercel Cron	Free on Hobby, 2/day limit — upgrade for 6hr schedule

Cost control tip: The diff step is your main guard. If you have 10 RSS feeds and each publishes 3 new items per run, and you have 20 flight books — that’s 60 Firecrawl scrapes × 20 OpenAI comparisons = 1,200 OpenAI calls per day max. Filter early by checking if the scraped content is even topically relevant before sending all 20 book comparisons.

Add a **relevance pre-filter** before the per-book loop in `route.ts` : use a cheap GPT-4o-mini call first to classify if the scraped content is aviation-relevant at all before running the full comparison against every book.

Schema version 1.0 – Build with Cursor